Aim: To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)

IDE: Google Colab

Theory:

To preprocess your text simply means to bring your text into a form that is predictable and analyzable for your task. A task here is a combination of approach and domain. Machine Learning needs data in the numeric form. We basically used encoding technique (Bag Of Word, Bi-gram, n-gram, TF-IDF, Word2Vec) to encode text into numeric vector. But before encoding we first need to clean the text data and this process to prepare (or clean) text data before encoding is called text preprocessing, this is the very first step to solve the NLP problems.

Tokenization:
Tokenization is about splitting strings of text into smaller pieces, or "tokens". Paragraphs can be tokenized into sentences and sentences can be tokenized into words.

Filtration:
Similarly, if we are doing simple word counts, or trying to visualize our text with a word cloud, stopwords are sonic of the most frequently occurring words but don't really tell us anything. We're often better off tossing the stopwords out of the text. By checking the Filter Stopwords option in the Text Pre-processing tool, you can automatically filter these words out.

Stemming:
Stemming is the process of reducing inflection in words (e.g. troubled, troubles) to their root form (e.g. trouble). The "root" in this case may not be a real root word, but just a canonical form of the original word.

Stemming uses a crude heuristic process that chops off the ends of words in the hope of correctly actually be converted to troublinstead of trouble because the ends were just chopped off (ughh, how crude!). There are different algorithms for stemming. The most common algorithm, which is also known to be empirically effective for English, is Porters Algorithm. Here is an example of stemming in action with Porter Stemmer:

	original word	**stemmed words**
0	connect	connect
'I	connected	connect
2	connection	connect
3	connections	connect
4	connects	connect

Stopword Removal
Stop words are a set of commonly used words in a language. Examples of stop words in English are "a", ‘‘the", "is", "are" and etc. The intuition behind using stop words is that, by removing low information words from text, we can focus on the important words instead.

For example, in the context of a search system, if your search query is 'what is text preprocessing?", you want the search system to focus on surfacing documents that talk about text preprocessing over documents that talk about what is. This can be done by preventing all words from your stop word list from being analyzed. Stop words are commonly applied in search systems, text classification applications, topic modeling, topic extraction and others. Stop word removal, while effective in search and topic extraction systems, showed to be non-critical in classification systems. However, it does help reduce the number of features in consideration which helps keep your models decently sized

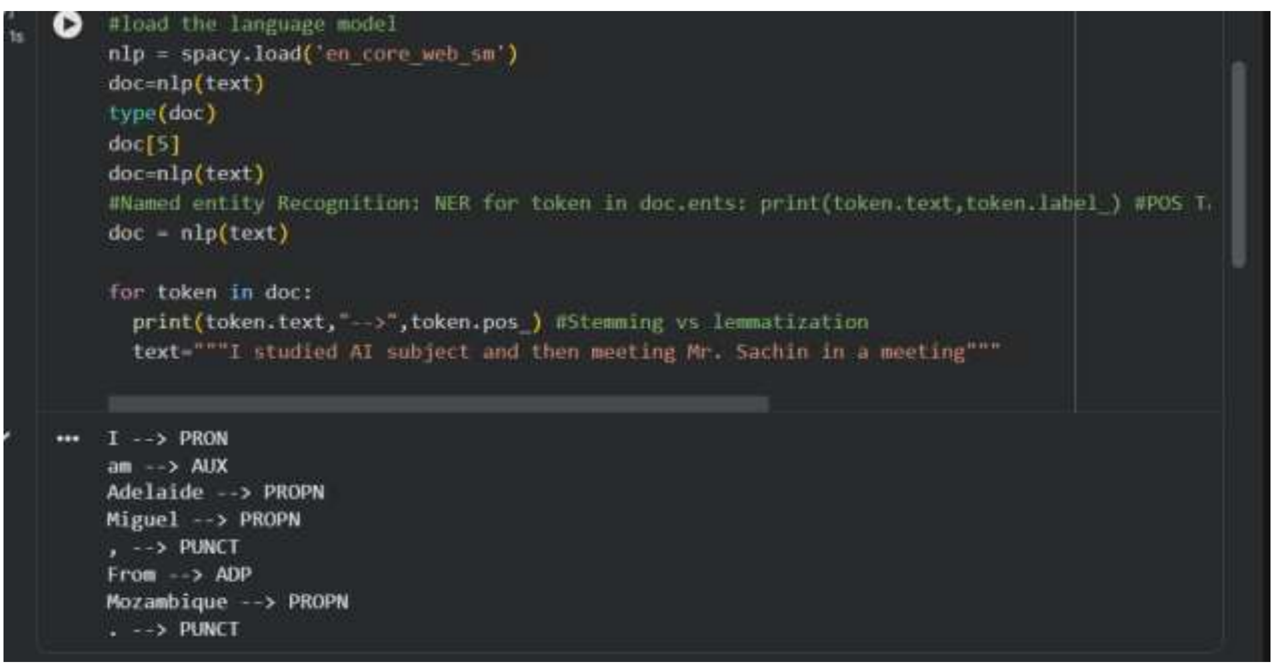
Program (Code):

```
text = "I am Adelaide Miguel, From Mozambique."
type(text)
text[5]
#NLP : SPACY,NLTK, Gensim
import spacy
#load the language model
nlp =
spacy.load('en_core_web_sm')
doc=nlp(text)
type(doc)
doc[5]
doc=nlp(text)
#Named entity Recognition:
NER for token in doc.ents:
print(token.text,token.label_)
#POS Tags: Part of Speech
tagging text = "Let's google it"
doc = nlp(text)
for token in
doc:
  print(token.text,"-->",token.pos_)
#Stemming vs lemmatization
text="""I studied AI subject and
then meeting Mr. Sachin in a meeting"""
#stemming
import nltk
```

```
from nltk.stem.porter import *

p_stem=PorterStemmer()
for word in text.split():
  print(word,"-->",p_stem.stem(word))
#lemmatization
for token in nlp(text):
  print(token.text,"->
  ",token.pos_,
      "->",token.lemma_)
```

Results:

```
text = "I am Adelaide Miguel, From Mozambique."

type(text)
text[5]

'A'
```



```
#load the language model
nlp = spacy.load('en_core_web_sm')
doc=nlp(text)
type(doc)
doc[5]
doc=nlp(text)
#Named entity Recognition: NER for token in doc.ents: print(token.text,token.label_) #POS T.
doc = nlp(text)

for token in doc:
  print(token.text,"-->",token.pos_) #Stemming vs lemmatization
  text="""I studied AI subject and then meeting Mr. Sachin in a meeting"""

I --> PRON
am --> AUX
Adelaide --> PROPN
Miguel --> PROPN
, --> PUNCT
From --> ADP
Mozambique --> PROPN
. --> PUNCT
```

```
I --> i
studied --> studi
AI --> ai
subject --> subject
and --> and
then --> then
meeting --> meet
Mr. --> mr.
Sachin --> sachin
in --> in
a --> a
meeting --> meet
I ->  PRON -> I
studied ->  VERB -> study
AI ->  PROPN -> AI
subject ->  NOUN -> subject
and ->  CCONJ -> and
then ->  ADV -> then
meeting ->  VERB -> meet
Mr. ->  PROPN -> Mr.
Sachin ->  PROPN -> Sachin
in ->  ADP -> in
a ->  DET -> a
meeting ->  NOUN -> meeting
```

Observation:

1. Tokenization

- The given text was successfully divided into smaller units called tokens (words).
- Each sentence was broken into individual words, making it easier to process.
- Tokenization helps in analyzing text at a granular level.

2. Stopword Removal (Filtration)

- Common words like *"is", "am", "the", "are"* were removed.
- This reduced unnecessary data and kept only meaningful words.
- It helps in improving efficiency and reducing feature size.

3. Stemming

- Words were reduced to their root form using the Porter Stemmer.
- Some stemmed words were not meaningful or not proper English words.
- Examples observed:

    studying → studi
    running → run
    intelligence → intellig

- Stemming is **fast but less accurate**.

1. Lemmatization

- Words were converted into their base (dictionary) form using linguistic rules.
- The output words were meaningful and grammatically correct.
- Examples observed:

  studying → studying (or study depending on context)
  running → run
  better → good
- Lemmatization is slower but more accurate than stemming.

2. Overall Observation

- Text preprocessing helps in cleaning and preparing text for NLP tasks.
- Stemming and lemmatization both reduce word forms, but:

  Stemming is quicker but less precise.
  Lemmatization gives better and meaningful results.
- Preprocessing improves the performance of machine learning models.

3. <u>Post Lab Exercise:</u>

```
# Install required libraries
!pip install nltk

nltk.download('punkt')
nltk.download('punkt_tab')   # ⭐ IMPORTANT (fixes your error)
nltk.download('stopwords')
nltk.download('wordnet')

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer

# Input text
text = """I am studying Artificial Intelligence at my university.
She loves reading books and writing blogs.
The students are preparing for their exams.
He was running very fast in the competition.
We are learning natural language processing.
They have completed their assignments successfully.
The teacher is explaining the concepts clearly.
I enjoy solving challenging problems.
She is practicing coding every day.
Machines are becoming smarter with data."""

# Tokenization
tokens = word_tokenize(text)
print("Tokens:\n", tokens)

# Stopword Removal
stop_words = set(stopwords.words('english'))
```

Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:**	**Enrolment No : 92301733009**

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.3)
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
Tokens:
 ['I', 'am', 'studying', 'Artificial', 'Intelligence', 'at', 'my', 'university', '.', 'She', 'loves', 'reading', 'books', 'and', 'writing', 'blogs', '.', 'The', 'students', 'are', 'preparing', 'for', 'their', 'exams', '.', 'He', 'was', 'runn

After Stopword Removal:
 ['studying', 'Artificial', 'Intelligence', 'university', '.', 'loves', 'reading', 'books', 'writing', 'blogs', '.', 'students', 'preparing', 'exams', '.', 'running', 'fast', 'competition', '.', 'learning', 'natural', 'language', 'processing

Stemmed Words:
 ['studi', 'artifici', 'intellig', 'univers', '.', 'love', 'read', 'book', 'write', 'blog', '.', 'student', 'prepar', 'exam', '.', 'run', 'fast', 'competit', '.', 'learn', 'natur', 'languag', 'process', '.', 'complet', 'assign', 'success', '

Lemmatized Words:
 ['studying', 'Artificial', 'Intelligence', 'university', '.', 'love', 'reading', 'book', 'writing', 'blog', '.', 'student', 'preparing', 'exam', '.', 'running', 'fast', 'competition', '.', 'learning', 'natural', 'language', 'processing', '.
```